

Lección 26: Heuristics & Accessibility

Diseño Usable e Inclusivo

Agenda

- Heurísticas de Usabilidad (Nielsen)
 - Fundamentos de Accesibilidad (A11Y)
 - WCAG y niveles de cumplimiento
 - Testing de accesibilidad
 - Mejores prácticas
-

¿Por qué Usabilidad? (1/2)

El costo de mala usabilidad:

Flujo de usuario sin usabilidad:

1. Usuario llega al sitio
2. Confundido por la interfaz
3. No encuentra "Agregar al Carrito"
4. Hace click en botón equivocado
5. Mensaje de error poco claro
6. Se rinde
7. Compra a la competencia

Tasa de conversión: 0%

¿Por qué Usabilidad? (2/2)

Con buena usabilidad:

Flujo de usuario con usabilidad:

1. Usuario llega al sitio
2. Navegación clara
3. Botón "Agregar al Carrito" obvio
4. Confirma la acción
5. Mensaje de éxito claro
6. Checkout fácil
7. Completa la compra

Tasa de conversión: 12%

Usabilidad = Ingresos

10 Heurísticas de Usabilidad de Nielsen

Jakob Nielsen (1994) - Aún relevantes hoy

1. Visibilidad del estado del sistema
 2. Coincidir con el mundo real
 3. Control y libertad del usuario
 4. Consistencia y estándares
 5. Prevención de errores
 6. Reconocimiento antes que recuerdo
 7. Flexibilidad y eficiencia de uso
 8. Diseño estético y minimalista
 9. Ayudar a reconocer, diagnosticar y recuperarse de errores
 10. Ayuda y documentación
-

Heurística 1: Visibilidad del Estado

Mantener usuarios informados de lo que sucede

-  MAL: Click "Agregar al Carrito" → Nada pasa por 2 segundos
-  BIEN: Click → Spinner "Agregando..." → ✓ "¡Agregado!"

En el carrito:

- Spinner al cargar productos
- Badge con cantidad de items ( 3)
- "Guardando..." al actualizar cantidad

Heurística 2: Lenguaje del Usuario

Hablar como habla el usuario, no como programador

✗ MAL: "Persistir ítem en repositorio de compras"

✓ BIEN: "Agregar al Carrito"

✗ MAL: "Inicializar flujo de checkout"

✓ BIEN: "Ir a Pagar"

✗ MAL: "1999 centavos"

✓ BIEN: "\$19.99"

Usar símbolos universales:  (carrito),  (eliminar)

Heurística 3: Control y Libertad

Permitir deshacer acciones

✗ MAL: Borrar ítem inmediatamente (sin confirmación)

✓ BIEN: Confirmación + botón "Undo" por 5 segundos

En el carrito:

- Confirmar antes de "Clear cart"
- Toast con "Undo" al remover ítem
- Cancelar checkout en cualquier momento

Heurística 4: Consistencia

Mismo comportamiento = misma apariencia

✗ MAL: 3 estilos diferentes de botones
`<Button variant="primary">Agregar</Button>`
`<button className="btn-blue">Pagar</button>`
`<a className="link-button">Continuar</a>`

✓ BIEN: Mismo componente en todo el sitio
`<Button variant="primary">Agregar al Carrito</Button>`
`<Button variant="primary">Pagar</Button>`

- Colores significan lo mismo en todas partes
- Terminología consistente ("Carrito", nunca "Cesta")

Heurísticas 5-7 (Resumidas)

5. Prevención de Errores: Deshabilitar acciones inválidas

- ✓ Botones [- 2 +] (imposible escribir "abc" o "-5")
- ✓ "Checkout" disabled si cart vacío
- ✓ Submit disabled mientras loading

6. Reconocimiento > Recuerdo: Mostrar info, no forzar a recordar

- ✓ Toast muestra imagen + nombre del producto agregado
- ✓ Cart muestra precio (no solo ID del producto)

7. Flexibilidad: Para novatos y expertos

- ✓ Mouse: Click botones
- ✓ Teclado: Tab + Enter (shortcuts opcionales)

Heurísticas 8-10 (Resumidas)

8. Diseño Minimalista: Mostrar solo lo esencial

✗ MAL: SKU, Categoría, Peso, Dimensiones, Color...

✓ BIEN: Nombre, Imagen, Precio, Cantidad [- 2 +]

9. Recuperación de Errores: Mensajes claros con soluciones

✗ MAL: "Error: INVALID_INPUT_VALIDATION_FAILED_422"

✓ BIEN: "Producto sin stock. [Notificarme cuando esté disponible]"

10. Ayuda Contextual: UI auto-explicativa + tooltips si es necesario

✓ Placeholder: "VERANO20" (muestra formato)

✓ Tooltip: ⓘ "Ingresa código de descuento del email"

Accesibilidad Web (A11Y)

¿Qué es accesibilidad?

Hacer aplicaciones web usables por TODOS, incluyendo:

- Discapacidades visuales (ciegos, baja visión, daltonismo)
- Discapacidades motoras (no pueden usar mouse)
- Discapacidades auditivas (sordos, hipoacusia)
- Discapacidades cognitivas (dislexia, TDAH)
- Discapacidades temporales (brazo roto, luz solar intensa)

Por qué importa:

- 15% de la población mundial tiene discapacidades
- Requisito legal (ADA, Section 508)
- Mejor UX para todos
- Beneficios SEO
- Imperativo ético

WCAG: Guías de Accesibilidad de Contenido Web (1/2)

4 Principios (POUR):

1. Perceptible

- Alternativas de texto para imágenes
- Subtítulos para videos
- Contraste de color suficiente
- Texto redimensionable

2. Operable

- Accesible por teclado
- Tiempo suficiente para leer
- Sin destellos que causen convulsiones
- Navegación clara

WCAG: Guías de Accesibilidad de Contenido Web (1/2)

3. Comprensible

- Texto legible
- Comportamiento predecible
- Asistencia de entrada
- Identificación de errores

4. Robusto

- Compatible con tecnologías asistivas
- HTML válido
- Marcado semántico

Niveles de Accesibilidad

Niveles WCAG:

Nivel A: Mínimo (básico)

Nivel AA: Rango medio (objetivo para la mayoría de sitios)

Nivel AAA: Más alto (no requerido para todo el contenido)

Objetivo: WCAG 2.1 Nivel AA

Consecuencias de no cumplir:

- Demandas (Domino's Pizza perdió demanda de \$4M)
 - Clientes perdidos (15% de la población)
 - SEO pobre (Google considera accesibilidad)
 - Daño a la reputación
-

HTML Semántico

No semántico:

```
<div onClick={handleClick}>Haz click</div>
```

Problemas:

- Lectores de pantalla no saben que es clickeable
- No navegable por teclado
- Sin indicación de foco
- Sin estados por defecto

Semántico:

```
<button onClick={handleClick}>Haz click</button>
```

Beneficios:

- Lectores de pantalla anuncian "botón"
- Navegación Tab funciona
- Foco visible
- Enter/Espacio lo activan

Etiquetas ARIA

Cuando HTML semántico no es suficiente:

```
// Botones con íconos (lectores de pantalla no saben qué significa)
<button aria-label="Eliminar ítem del carrito">
  <TrashIcon />
</button>

// Contenido dinámico (anuncia cambios)
<div aria-live="polite">
  {itemCount} ítems en el carrito
</div>

// Estados de carga
<button aria-busy={isLoading} disabled={isLoading}>
  {isLoading ? 'Agregando...' : 'Agregar al Carrito'}
</button>
```

Contraste de Color

Requisitos WCAG AA:

Texto normal: contraste 4.5:1
Texto grande (18pt+): contraste 3:1

Ejemplos:

✗ Contraste insuficiente (gris claro en blanco: 2:1)
color: **#999999**;
background: **#FFFFFF**;

✓ Contraste suficiente (gris oscuro en blanco: 7:1)
color: **#595959**;
background: **#FFFFFF**;

Herramientas:

- Chrome DevTools: Inspeccionar elemento → Ratio de contraste
 - WebAIM Contrast Checker
 - Axe DevTools (extensión de navegador)
-

Navegación por Teclado y Foco

TODO debe ser accesible sin mouse:

```
// ✅ Usar elementos nativos (accesibles por defecto)
<button onClick={handleClick}>Eliminar</button> // Tab, Enter, Espacio

// ❌ NUNCA remover outline de foco sin alternativa
button:focus { outline: none; } // ❌

// ✅ Foco visible personalizado
button:focus-visible {
  outline: 3px solid #4A90E2;
  outline-offset: 2px;
}
```

Trampa de foco en modales: El foco no puede escapar del modal (usar librería como `focus-trap-react`)

Test: Navegar app completa con Tab/Shift+Tab/Enter/Escape

Testing de Accesibilidad

Herramientas Automáticas:

- Axe DevTools (Chrome extension)
- Lighthouse (Chrome DevTools → Accessibility score)
- WAVE (browser extension)

Testing Manual (REQUERIDO):

1. Keyboard navigation (Tab/Enter/Escape sin mouse)
2. Screen readers:

- macOS: VoiceOver (Cmd + F5)
 - Windows: NVDA (gratis)
3. Color contrast (Chrome DevTools)
 4. Zoom 200% (texto debe ser legible)

Checklist rápido

- ☒ Imágenes con alt text
- ☒ Botones con labels
- ☒ Forms con labels
- ☒ Contraste 4.5:1
- ☒ Keyboard accessible

Ejercicio 1: Agregar ARIA Labels para Accesibilidad

Prompt:

Actúa como un accessibility engineer implementando ARIA labels en component

CONTEXT0: Screen readers (lectores de pantalla) = herramientas usadas por p
o con baja visión (VoiceOver en macOS, NVDA/JAWS en Windows, TalkBack en An
de la población mundial tiene alguna discapacidad. WCAG 2.1 Level AA = está
requerido (ADA, Section 508). Botones con solo íconos: screen readers anunc
sin contexto (usuario NO sabe qué hace). aria-label attribute: proporciona
accesible para elementos sin texto visible. Template literals: permiten con
dinámico en labels (ej: `\${itemCount} items`). Chrome DevTools Accessibilit
Computed Properties → Name muestra qué lee screen reader.

TAREA: Agrega aria-label descriptivo a botón con ícono de carrito en CartSu

UBICACIÓN:

- Archivo: src/features/shopping-cart/components/CartSummary.tsx
- Componente: Botón "View cart" que solo muestra ícono de carrito (sin text
- Problema actual: Screen reader anuncia solo "Button" (NO dice qué hace)

IMPLEMENTACIÓN ARIA-LABEL:

- Atributo: aria-label con descripción completa de la acción
- Contenido dinámico: Incluir itemCount en el label usando template literal

- Formato: `View shopping cart with `${itemCount}` items`
- Ejemplo: Si itemCount = 3 → "View shopping cart with 3 items"

PATRÓN DE CÓDIGO:

```
<button
  onClick={handleViewCart}
  aria-label={`View shopping cart with ${itemCount} items`}
>
  <CartIcon />
</button>
```

TESTING CON CHROME DEVTTOOLS:

1. Inspeccionar botón (click derecho → Inspect)
2. Abrir panel Accessibility (en DevTools tabs)
3. Verificar sección "Computed Properties"
4. Confirmar Name = "View shopping cart with X items" (NO solo "Button")

TESTING CON SCREEN READER (macOS):

1. Activar VoiceOver: Cmd + F5
2. Navegar al botón con Tab
3. Escuchar anuncio: debe decir "View shopping cart with 3 items, button"
4. Desactivar VoiceOver: Cmd + F5

VALIDACIÓN: Screen reader debe anunciar nombre completo con contexto, NO so

Aprende: ARIA labels hacen interfaces accesibles

proporcionando contexto a screen readers

Ejercicio 2: Auditar y Mejorar Keyboard Navigation

Prompt:

Actúa como un accessibility auditor verificando keyboard navigation en la a

CONTEXT0: Keyboard accessibility = WCAG 2.1 Level A requerido (legal). Usua discapacidades motoras, usuarios de tecnologías asistivas, power users usan teclado (NO mouse). Tab key: navega elementos interactivos en orden. Shift+ navega hacia atrás. Enter/Space: activa botones. Escape: cierra modals/drop Focus visible: outline/ring visual que muestra elemento activo (NUNCA usar none sin alternativa). tabIndex={0}: hace elemento focusable en orden natur

tabIndex={-1}: focusable programáticamente pero NO en Tab order. focus-visible pseudo-class: muestra focus SOLO cuando usuario usa teclado (NO con mouse)

TAREA: Audita keyboard navigation en carrito y corrige problemas encontrados

TESTING CHECKLIST (usar SOLO teclado):

1. Tab Order: Verifica orden lógico (top→bottom, left→right)
 - Desde logo → productos → botones "Add to cart" → carrito
 - Skip to main content link debe ser primer elemento focusable
2. Interactive Elements: Todos deben ser accesibles con teclado
 - Botones "Add to cart"
 - Controles de cantidad (+/-)
 - Botón "Remove item"
 - Botón "Checkout"
3. Keyboard Actions: Verificar teclas funcionan
 - Enter/Space en botones → ejecuta acción
 - Escape en modals → cierra modal
 - Arrow keys en inputs numéricos → incrementa/decrementa
4. Focus Visibility: Debe haber outline/ring visible en TODOS los elementos
 - Si NO hay outline visible → agregar estilos focus-visible

PROBLEMAS COMUNES A BUSCAR:

- Elementos clickables que NO son <button> (usar div con onClick)
- outline: none sin alternativa de focus styling
- Orden de Tab ilógico (elementos fuera de secuencia visual)
- Modals que NO trapean focus (focus escapa del modal)
- Skip to main content link faltante

SOLUCIONES A IMPLEMENTAR:

Para focus visibility (si falta):

```
/* Agregar a index.css o global styles */
button:focus-visible,
a:focus-visible,
input:focus-visible {
  outline: 3px solid #4A90E2;
  outline-offset: 2px;
}
```

Para elementos no focusables (si usan div en vez de button):

```
// ❌ NO accesible
<div onClick={handleClick}>Click me</div>
```

```
//  Accesible  
<button onClick={handleClick}>Click me</button>
```

VALIDACIÓN FINAL:

1. Desconectar mouse físicamente (o NO tocarlo)
2. Navegar TODA la aplicación con Tab/Shift+Tab/Enter/Escape
3. Verificar que TODOS los elementos interactivos son accesibles
4. Confirmar focus visible en todo momento (saber dónde estás)
5. Verificar orden lógico de navegación

VALIDACIÓN: Poder usar toda la app sin mouse, con focus siempre visible

Aprende: Keyboard navigation no es opcional -

es requerimiento legal de accesibilidad

Puntos Clave

1. **Heurísticas de Nielsen:** 10 principios para diseño usable
2. **Estado del Sistema:** Siempre informar a usuarios qué está sucediendo
3. **Control del Usuario:** Permitir deshacer, confirmación para acciones destructivas
4. **Consistencia:** Mismos patrones en toda la app
5. **Prevención de Errores:** Validar input, deshabilitar acciones inválidas
6. **A11Y:** 15% de usuarios tienen discapacidades
7. **WCAG AA:** Estándar objetivo para accesibilidad
8. **HTML Semántico:** Usar elementos correctos (button, no div)
9. **Navegación por Teclado:** Todo accesible sin mouse
10. **Testing:** Herramientas automáticas + testing manual + lectores de pantalla